

DLD MILESTONE 3 REPORT

[Ayesha Mir (09886), Divya Lakhani (10267), Areesha Ashfaq (09879), Raheem Waseem Khan (09546)]

Introduction:

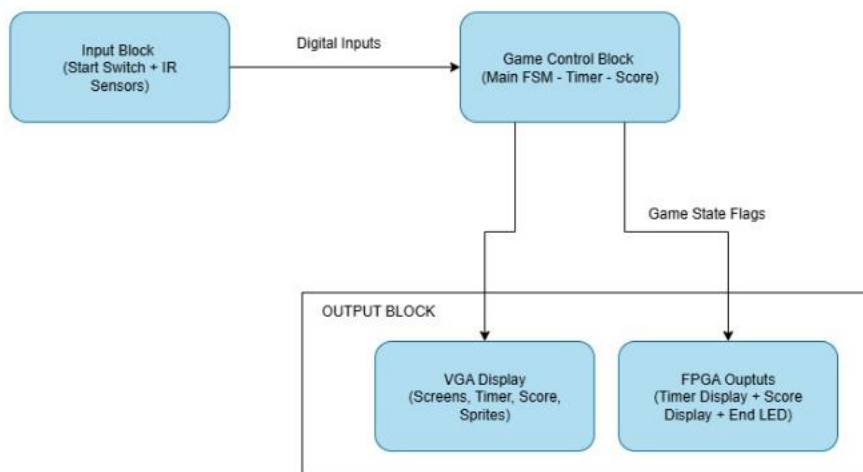
This project implements a Target shooting game like at carnivals called 'Point Blank'. It is implemented on the Basys-3 FPGA board. The primary objective of the system is to stimulate a real-time interactive game in which the player shoots down multiple targets within a limited time frame, with scoring and game progression handled through dedicated hardware logic.

The core components of the final system include a countdown timer, multiple active- high IR sensor interfaces for target detection, debouncing circuits, scoring logic, and game-over control. Gameplay progression is governed by a centralized control finite state machine (FSM) that explicitly tracks the system state through score-based states, enabling precise control over gameplay flow, score updates, and termination conditions. This FSM-based design provides clear separation between input conditioning, control logic, and output generation, while allowing the system to scale efficiently to multiple sensor inputs.

During gameplay, the system continuously monitors target states and updates the game status in real time. Target hits result in score updates and give immediate visual feedback, while the remaining time and current score are displayed dynamically on the VGA interface. The game proceeds until either all targets are hit, or the allotted time expires, at which point the system transitions into a game-over state and maintains the final results on the display.

The complete design was implemented and validated on physical FPGA hardware. All internal control logic, input conditioning, FSM transitions, sensor handling, and VGA output behavior were tested to ensure correct real-time operation. By integrating sensor-driven gameplay logic with dynamic VGA visualization, the project demonstrates the practical application of finite state machines, parallel sensor processing, and hardware-based graphics control in a real-time digital system.

System Block Diagram:



The system architecture consists of three modular blocks:

1. **Input Processing Block:** Processes all external inputs including the Start and the IR sensors' reading after debouncing. When Start = 1, we enter gameplay state and when start = 0, you're in idle/reset state
2. **Control Block:** Contains the main Finite State Machine (FSM) which governs the game sequence. This block includes the start delay logic, countdown timer, sensor handling, and score management. The timer is enabled after the start delay expires, while sensor inputs are monitored to detect valid hit events and update the score. Game-over conditions are asserted when the countdown timer reaches zero or when the maximum score of 60 is achieved.
3. **Output Processing Block:** Consumes the gamestate signals and internal registers to drive all player-visible outputs. This includes the VGA display system, which renders the start, gameplay, and game-over screens along with sprites, score, and timer text. Additionally, the seven-segment display presents the current score and remaining time, while an on-board LED provides a blinking indication when the game ends; the FPGA outputs are just for the team to follow the vga and debug easily.

Input Block:

The Input Block manages all external user inputs required for the game on the Basys-3 FPGA board. It interfaces directly with the Start toggle switch and the six IR sensor inputs, each corresponding to a target can in the game.

The Start switch is passed through a debounce module to generate a stable signal, ensuring reliable game control and preventing unintended transitions caused by mechanical switch noise. Each IR sensor signal is also debounced before further processing to remove noise and false triggers.

After debouncing, the IR sensor signals are monitored for rising-edge events (0→1 transitions), which indicate that a can has been successfully hit. These clean sensor signals are then forwarded to the game control logic, where they are used to update the score and mark the corresponding can as hit.

The Input Block outputs the debounced start signal and the cleaned IR sensor signals to the Game Control Logic Block, where they are used to enable the timer, control game flow, and determine game-over conditions.

```
19      wire start_clean;
20
21      debounce db_start (
22          .clock(clk100),
23          .unstableinputfromsensor(start),
24          .stable_out(start_clean)
25      );
26
27      //delay when we change screen

2
3      // Debounce all 6 sensors
4      generate
5          for (i = 0; i < NUM_SENSORS; i = i + 1) begin : sensor //dounces the 6 sesnors and does
6              debounce db (
7                  .clock(clock),
8                  .unstableinputfromsensor(sensor_raw[i]),
9                  .stable_out(sensor_clean[i])
10             );
11         end
12     endgenerate
13
```

Pin Configurations and I/O Details:

Peripheral	Basys-3 Pin	Signal Name	Description	Logic
Start	External	start_clean	Starts the game. Pass through the debounce module to	Active High

			remove switch noise.	
Reset	External	!start_clean	Resets game logic, score, and timer. Also debounced for stable transitions.	Active High
Sensors	B16, A16, B15, P18, K17, N17	sensor_clean	Detects can's presence. Rising edge after debounce = hit event.	Active High
Clock	W5	clk100	Drives debounce, FSM, timer and score logic	-

Output Block:

The Output Block is responsible for managing all the visual feedback related to the game's state. The outputs include a VGA display for real-time gameplay visualization, a score display, and a countdown timer. These outputs are received by signals generated from the Sensor, Score, Timer, and Game Control modules during gameplay.

The physical game setup consists of two shelves containing six cans arranged in two rows: three cans on the top shelf and three on the bottom shelf, separated by dividers. Each can position is monitored by a dedicated sensor above it. When a can is knocked off the shelf by the player's gun, the corresponding sensor detects the loss of the can and sends a trigger signal to the system. Each sensor can only contribute to the score once; after it has been triggered, it is latched and cannot increment the score again. The game ends when either all six cans have been hit (maximum score of 60 points) or the countdown timer expires







Timer Display:

The Timer module begins counting down from a predefined value of 30 seconds when the user flips the start switch. The remaining time is displayed on the VGA screen and updates in real time while the game is active. If the timer reaches zero before all cans are knocked down, the game immediately ends, the score stops updating, and the final results are displayed. The timer is disabled once the game ends. It is also displayed on the two right segments on the FPGA board for debugging.

```

18 end
19
20 always @(posedge clock) begin
21     if (reset) begin // agr reset switch on hai to sab initial stage pay wapis la jao
22         count <= 30;
23         running <= 0;
24         div <= 0;
25         done <= 0;
26     end
27     else if (switch && !done) //agr start switch on hai and timer khatam nhi hoa to start the timer
28         running <= 1;
29
30     if (running && !done && !stopwhenscore) begin //We only count if timer is running, khatam nhi hoa, and not stopped by score mod
31         if (div == 99_999_999) begin //100 MHz (100 million ticks per second), So if you count to 99,999,999, that takes 1 second
32             div <= 0; //Reset div to 0 after every second.
33             if (count > 0)
34                 count <= count - 1; // ye to simple count down horha
35             else begin // jab timer 0 hojata to done ko 1 dedo
36                 count <= 0;
37                 running <= 0;
38                 done <= 1;
39             end
40         end else
41             div <= div + 1; //agr 1 sec nhi hoa to inc divider
42     end
43     else
44         div <= 0; //reset
45 end
46 endmodule

```

Score Display:

The scoring system awards 10 points for each can that is successfully knocked off the shelf. The six sensors independently generate a trigger when their associated can is removed. Once all six sensors have been triggered, the score reaches its maximum value of 60 points.

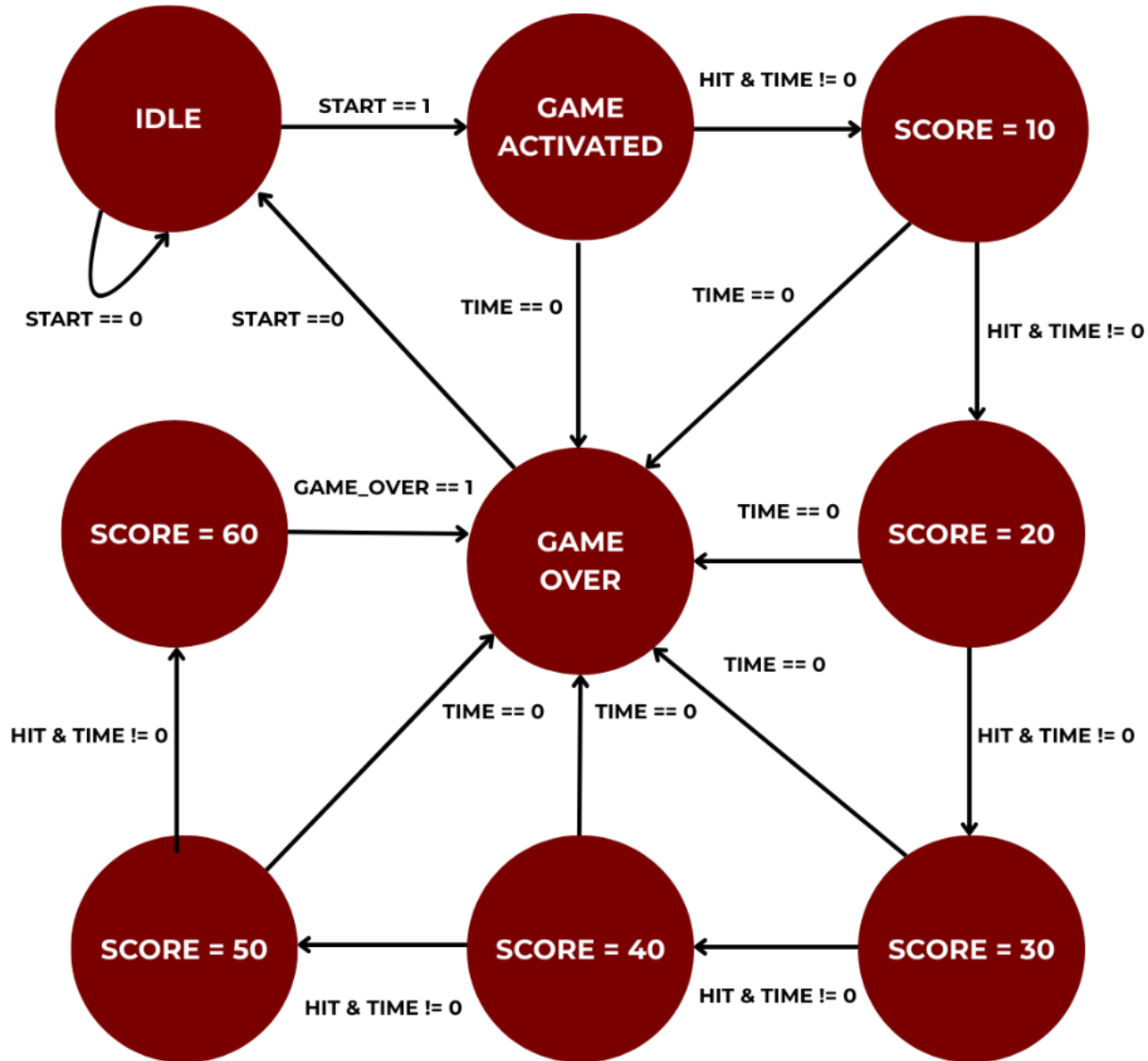
The current score is continuously sent to the display logic and shown to the player throughout the game. On the VGA display, the score is rendered numerically on-screen, providing real-time feedback. If the game ends, either due to the timer reaching zero or all six cans being hit, the score display freezes and continues to show the final score until the system is reset, and a new game is started using the start switch. It is also displayed on the two left segments on the FPGA board for debugging.

```

19 end
20
21 always @(posedge clock) begin
22     if (reset) begin //agr reset press to sab zeros
23         score <= 0;
24         prev_hit <= sensor_hit;
25         game_over <= 0;
26         locked_hits <= 0;
27     end
28     else if (!game_over) begin //iske nechay anything will only work if game_over is 0
29         if (timer_done) //agr timer mod se done 1 aya to game over
30             game_over <= 1;
31         else begin
32             if (running && sensor_hit && !prev_hit && locked_hits < 6) begin // agr pos edge
33                 //are diff and obv 60 se kam hai to score +10 and 6 mein 1 shot use hogya
34                 score <= score + 10;
35                 locked_hits <= locked_hits + 1;
36             end
37             prev_hit <= sensor_hit; //after that current state becomes prev state
38
39             if (score >= 60) begin //bas 60 tak jaiga if you reach it game over.
40                 score <= 60;
41                 game_over <= 1;
42             end
43         end
44     end
45 end
46 endmodule
47

```


Control Block:



In the final implementation, the Control Block is implemented using a formal Finite State Machine (FSM) that defines the progression of gameplay. The final design uses named states and defines transitions to control system operation in a structured manner. The control logic is implemented as a Moore-style FSM, meaning the system's outputs are tied to the current state rather than the incoming signals.

The control FSM is driven by three primary inputs:

- Start - > initiates gameplay and enables the timer
- Hit -> generated by a debounced rising-edge transition from an IR sensor
- Time==0 -> asserted by the timer that countdown expires

These inputs govern transitions between the FSM states as follows:

- When the system is in the IDLE state, all sensors are disabled, and the timer and score are reset. The system remains in this state until the Start button is switched on.
- When Start is asserted, the FSM transitions to the Game Activated state, starting the countdown timer.
- During active gameplay, each valid hit causes the FSM to transition sequentially through score-based states (Score = 10, 20, 30, 40, 50, and 60). Each score state represents a stable gameplay condition where the updated score is held and displayed while the timer continues to decrement.
- From any score state, if a valid hit occurs while time is still available, the FSM advances to the next score state. If the timer reaches zero at any point, the FSM transitions directly to the Game Over state.
- When the FSM reaches the Score = 60 state, indicating that all targets have been successfully hit, the system also transitions into the Game Over state. After a reset is issued in the Game Over state, the FSM transitions back to the IDLE state, restoring all counters, clearing used sensors, and preparing the system for a new round.

In the Game Over state, gameplay is halted, sensor inputs are ignored, the final score, and timer values are preserved. The system remains in this state until the Reset button is pressed, at which point the FSM transitions back to the IDLE state and prepares the system for a new game session.

User Flow Diagram:

The user flow outlines the complete gameplay sequence from the system startup to game termination, focusing on the Start/Reset inputs, debounced IR-based hit detection, real-time scoring, and the 30-second countdown.

The interaction follows a clear pathway: User Inputs → Input Block → Control Block → Output Block. When the player turns the switch on, the debounced signal transitions the system from the IDLE state to the Game Activated state. The countdown timer begins

decrementing; the VGA display switches to the gameplay screen, and all IR sensors are enabled. During this phase, the system continuously monitors the sensors for valid hit events while tracking the remaining time.

During the active phase, the system continuously monitors all IR sensors, where a debounced rising-edge (0→1) transition indicates that a target has been hit. Each valid hit increments the score by 10 points, and the updated score along with the remaining time is displayed in real time on the VGA output. Gameplay continues while the countdown timer has not expired, and the total score has not reached the maximum value of 60.

When either condition is met, the Control Block asserts the game-over signal and freezes the outputs. Turning the switch to reset state returns all modules to their initial IDLE state by resetting the score and timer, disabling the sensors, and restoring the VGA display to the home screen, thereby completing the user flow and preparing the system for a new game session.

